Medium Search Play Games Write

Jev | Artificial Intelligence | Frontier Tech | Valyu | Deep Research

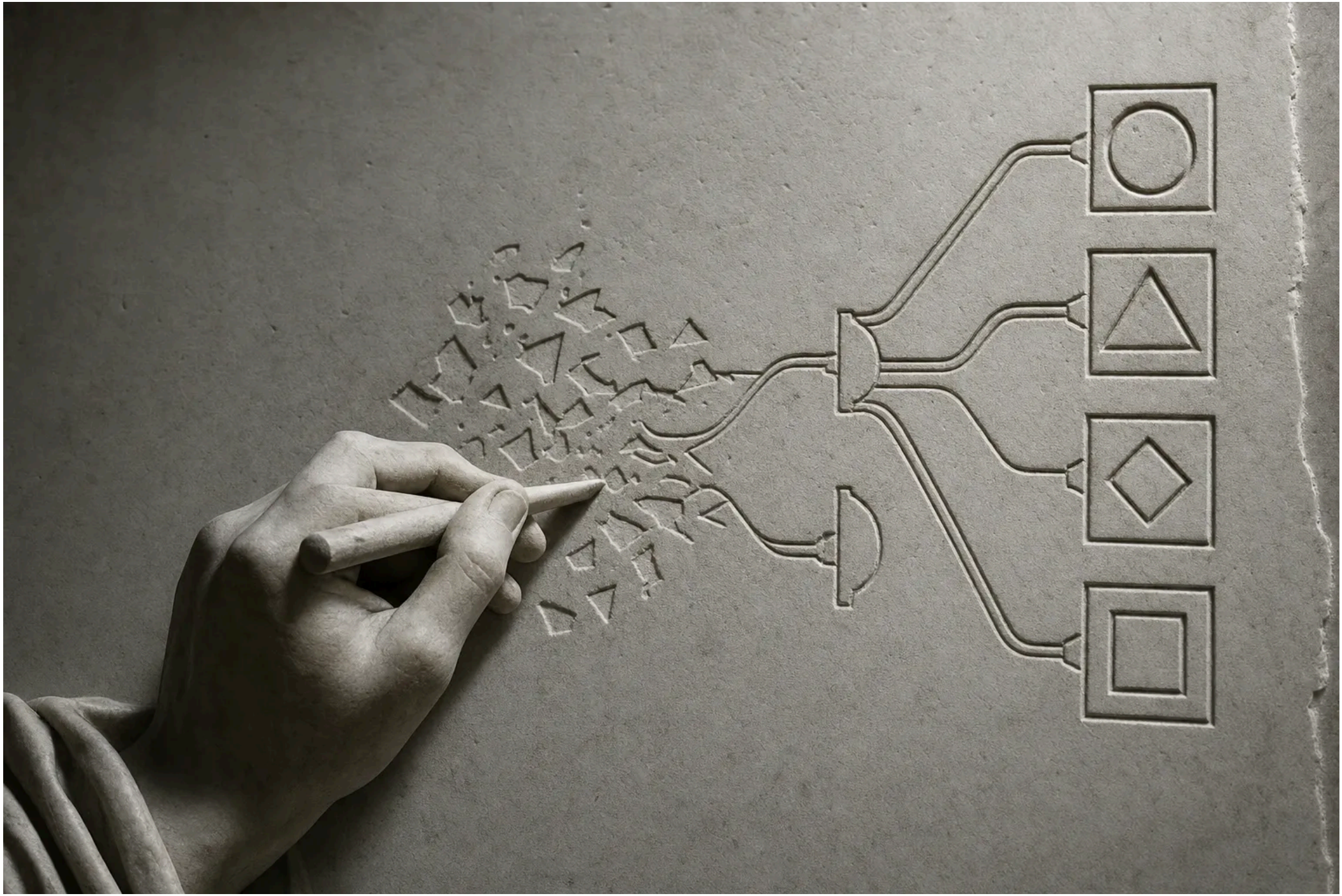
The Ultimate Guide to Jev: The new Frontier AI for faster decisions



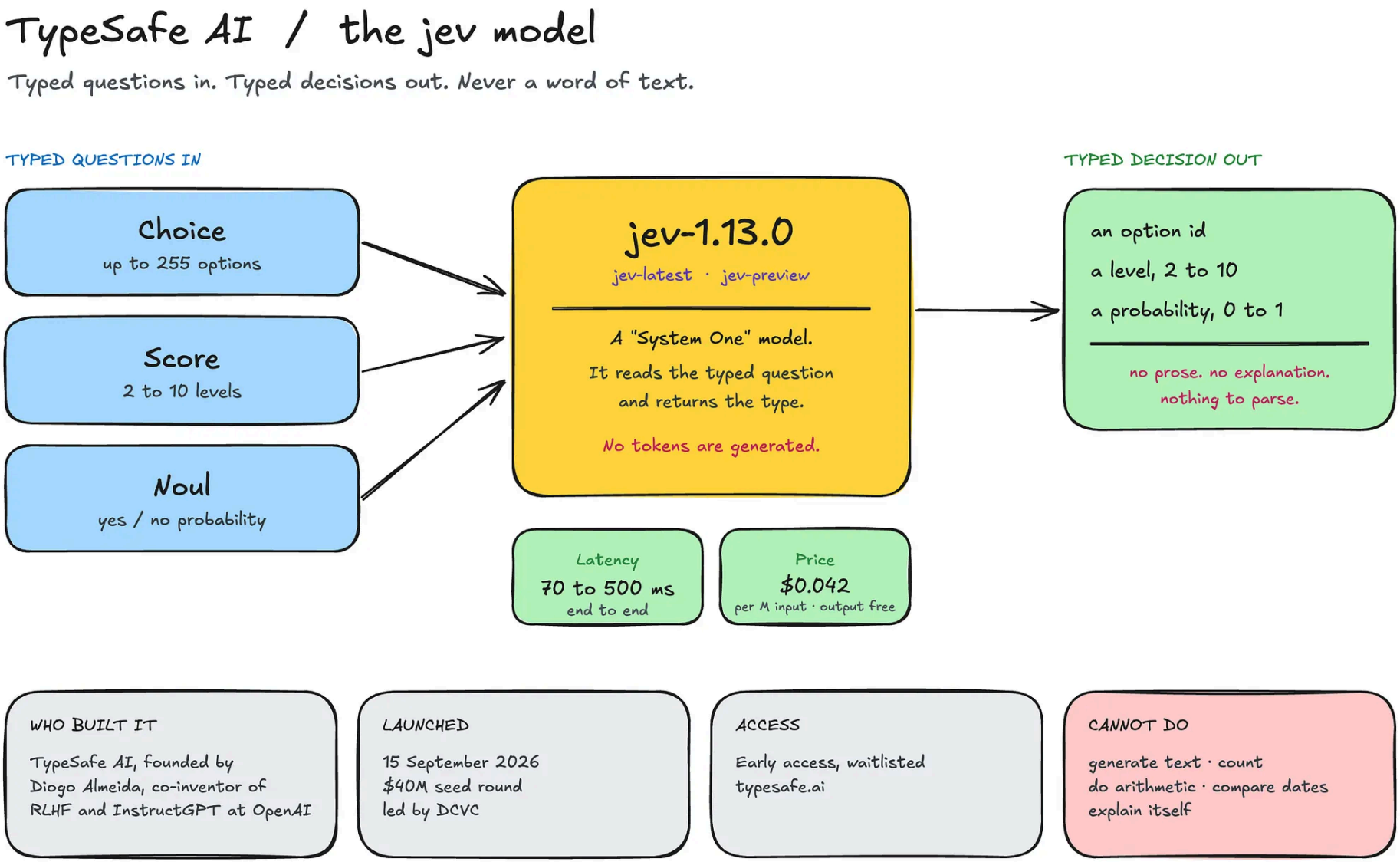
unicodeveloper Following 12 min read · Sep 17, 2026

1.5K 28 19

ChatGPT co-creator spent two years building Jev. Here is what it does, when to use it, when not to and what people have already built with it.



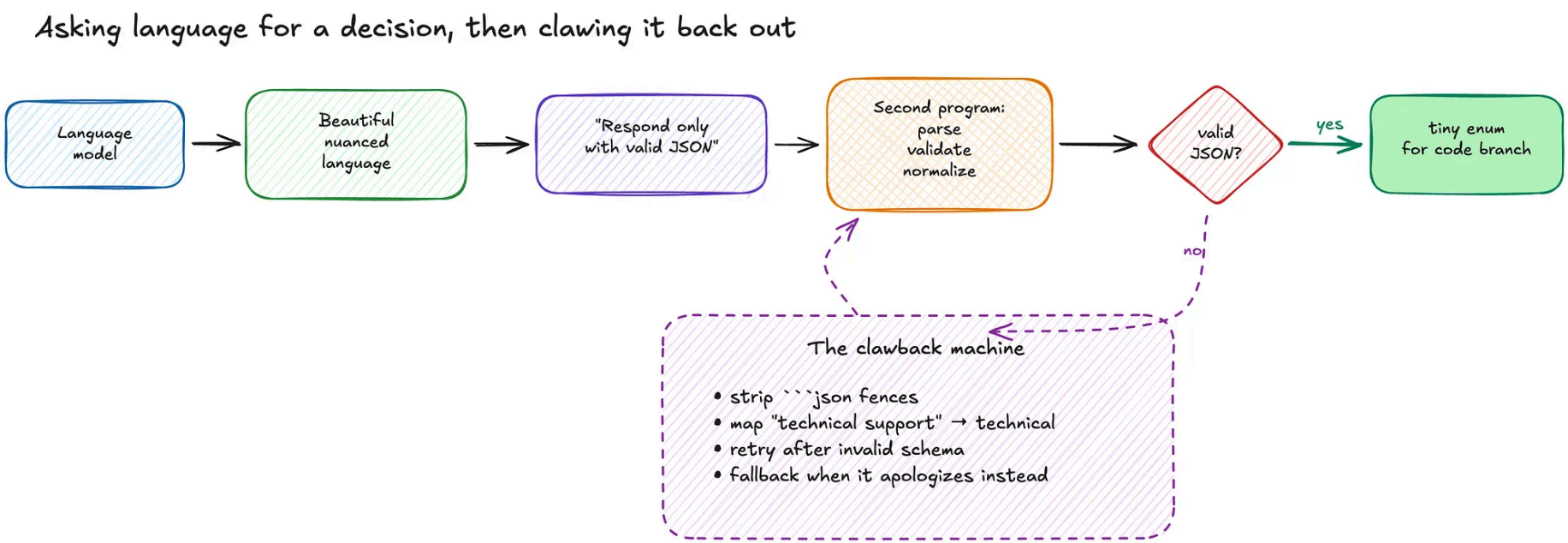
Quick answer: Jev is a frontier AI model from TypeSafe AI that returns typed, probabilistic decisions instead of generated text. You send it program state and typed questions; it answers all of them in a single parallel pass in 70 to 500 milliseconds, at $0.042 per million input tokens with output free. It cannot produce a malformed value, because you define the answer space before you ask. It launched two days ago, out of two years of stealth!



What Exactly does Jev Solve?

You have a model that understands language and need a decision your code can branch on.

Ask the model for language, then you write a second program whose entire job is to claw a decision back out of the language. You prompt it to “respond only with valid JSON.” You parse. You validate. You catch the case where it wrapped the object in a markdown fence. You retry when the enum came back as `"technical support"` instead of `technical`. You write a fallback for when it apologises instead of answering.



A model understood the intent. The surrounding code rebuilds certainty from prose after the fact.

That second program is not a small thing. In most production AI systems it is a meaningful share of the code, and nearly all of the flakiness.

Diogo Almeida (the founder) said this:,

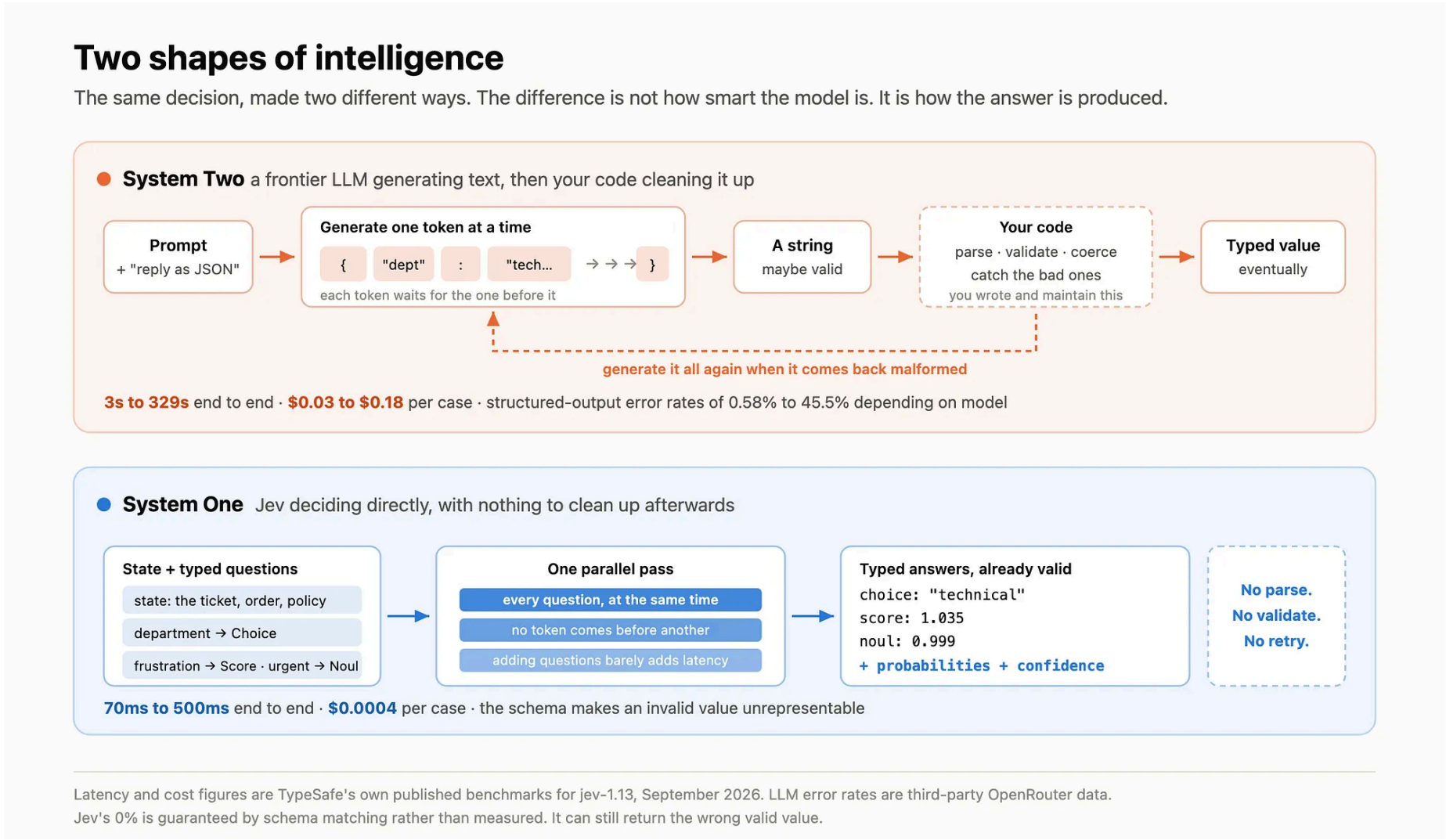
> “At OpenAI, I helped build the methods that made language models useful at following instructions and talking with people. That work ended up as the research behind ChatGPT. At the time, I thought maybe chat models would lead to AGI, but despite the hype it became obvious to me that there was something really big missing.”

The person who taught models to talk to people has spent two years building one that talks only to software.

What is a System One model?

A System One model is a class of AI model that makes fast, structured decisions software can use directly, returning typed values and calibrated probabilities instead of generated text.

System 2 is slow, deliberate reasoning. System 1 is the fast judgment you make before you have finished reading the sentence.



An LLM generates tokens one at a time, then your code parses, validates and retries. Jev evaluates every question at once and returns values that are already valid.

Almost every frontier lab is racing to build better System 2. Meanwhile most of the decisions inside real software are System 1, and we have been paying System 2 prices for them.

The architectural difference is not a matter of model size. An LLM produces its answer one token at a time, each token conditioned on the last, which is why latency scales with output length and why the answer arrives as a string that might be anything. Jev takes your state and all of your questions, evaluates them in a single parallel pass, and returns typed values drawn from an answer space you defined in advance.

The name is a nod to William Stanley Jevons, whose paradox holds that when something gets dramatically cheaper, we use dramatically more of it. TypeSafe is betting that intelligence at a two-hundredth of the price is not the same product at a discount. It is a different product, because it makes an entire class of decisions worth automating that previously were not.

Jev vs a frontier LLM

Two different machines

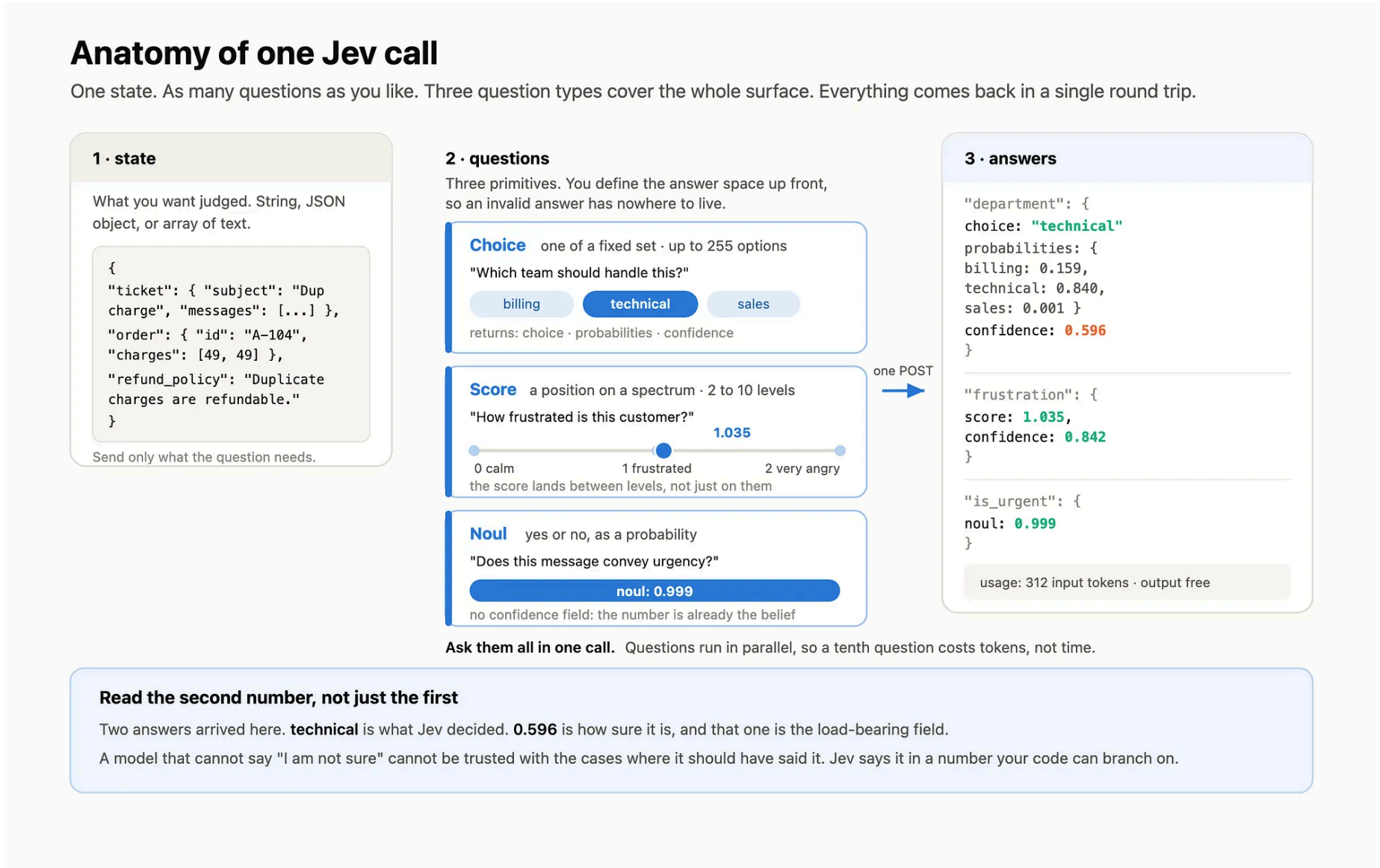
System One decides. System Two writes. They are not competing for the same job.

	jev System One	Frontier LLM System Two
Output	Typed values, defined in advance	A string you must parse and validate
Sampling	All outputs in one parallel pass	One token at a time
Latency	70 ms to 500 ms	3 s to 329 s
Input price	$0.042 / MTok	$0.20 to $10 / MTok
Output price	Free	Roughly 5x input
Trained with	RLCD (calibrated decisions)	RLHF / RLVR
Uncertainty	Calibrated confidence on every answer	Overconfident when asked
Type errors	Impossible by construction	0.58% to 45.5%, by model
Can write prose	No	Yes

Everything jev gives up is text. Everything it buys back is a type you never have to check.

How do you use Jev?

You send a state (the thing to be judged) and a set of questions. There are three question types and no others. Choice picks one option from a set you define, up to 255. Score rates the state against two to ten ordered levels you describe in words. Noul answers a yes-or-no question with the probability that the answer is yes.



One state, as many questions as you like, and a response whose second number, confidence, is the load-bearing one.

Here is a support ticket being triaged:

```
from typesafe_sdk import Choice, Noul, Score, TypeSafeClient

client = TypeSafeClient()
ticket = (
    "Hi, I've been trying to connect my Stripe account for 3 days "
    "and it keeps failing. I'm losing sales. Please help ASAP."
)
response = client.system_one(
    state=ticket,
    questions={
        "department": Choice(
            instructions="Which team should handle this",
            criteria={
                "billing": "Payment or subscription issues",
                "technical": "Bugs or integration problems",
                "sales": "Pricing or account questions",
            },
        ),
        "frustration": Score(
            instructions="How frustrated the customer appears",
            criteria=[
                "Calm, just stating facts",
                "Frustrated but civil",
                "Very angry, strong language",
            ],
        ),
        "is_urgent": Noul(
            instructions="The message conveys urgency or time-sensitivity",
```

```
            ),
        },
    )

    print(response.answers["department"].choice)   # "technical"
    print(response.answers["frustration"].score)   # 1.035
    print(response.answers["is_urgent"].noul)      # 0.999
```

Three judgments, one round trip, roughly 400 milliseconds. No prompt engineering about output format, because the format is not something the model chooses. No `json.loads` wrapped in a try block.

Two details in that output are worth slowing down for. The frustration score came back as **1.035**, not 1. Score returns a position on your scale rather than a label, so a customer slightly past “frustrated but civil” reads as slightly past it. You get resolution without inventing a ten-point rubric.

And every Choice and Score answer carries a `confidence` value alongside the full probability distribution. That field is the one most people will underuse, and it is the most important thing in the response.

Here’s another example using Valyu for clinical research and using Jev.

Valyu is the one of the best AI search & deepresearch platforms for academic research, financial and life sciences deepresearch work.

```
from valyu import Valyu
from typesafe_sdk import Noul, Score, TypeSafeClient

valyu = Valyu()          # reads VALYU_API_KEY
jev = TypeSafeClient()   # reads TYPESAFE_API_KEY

# 1. Retrieval: primary sources, filtered before anything reaches the model.
hits = valyu.search(
    "GLP-1 receptor agonists cardiovascular outcomes",
    included_sources=["valyu/valyu-pubmed", "valyu/valyu-arxiv"],
    start_date="2024-01-01",
    max_num_results=20,
    relevance_threshold=0.5,
)

# 2. Judgment: one bounded call per paper, roughly $0.0004 each.
shortlist = []
for paper in hits.results:
    verdict = jev.system_one(
        state={"title": paper.title, "source": paper.url, "content": paper.conte
        questions={
            "is_rct": Noul(
                instructions="This paper reports a randomised controlled trial")
            "reports_mace": Noul(
                instructions="The paper reports major adverse cardiovascular eve
            "evidence_strength": Score(
                instructions="How strong is the causal evidence presented",
                criteria=["Anecdotal or preclinical",
                          "Observational",
```

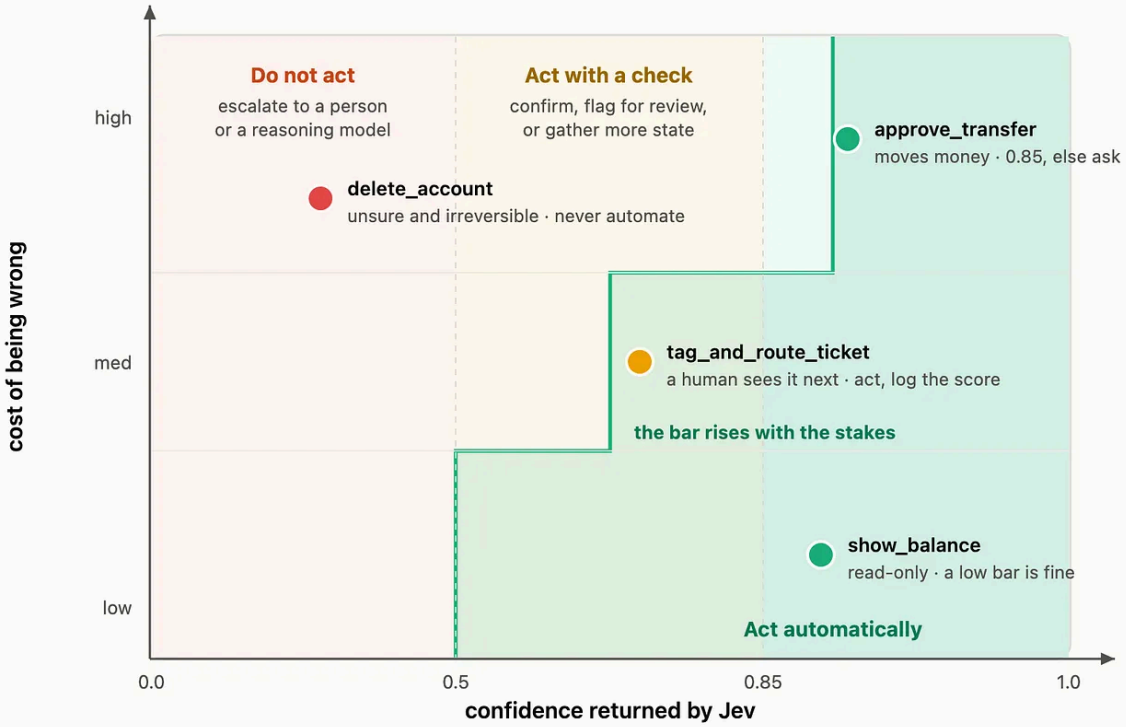
```
                        "Single randomised trial",
                        "Meta-analysis of randomised trials"],
            ),
        },
    )
    a = verdict.answers
    if a["is_rct"].noul > 0.7 and a["evidence_strength"].score > 1.5:
        shortlist.append((paper, a["evidence_strength"].confidence))
```

Why calibrated confidence changes how you design systems

Jev returns an answer and a number describing how much that answer should be trusted, and because of how the model was trained, the number means something.

Confidence is the second axis

The answer tells you what. Confidence tells you whether to act on it. One threshold for the whole system is the mistake.



what that looks like in code

```
action = response.answers["intent"]

# floor: below this, nobody acts
if action.confidence < 0.5:
route_to_human(msg)

# cheap to undo, low bar
elif action.choice == "check_balance":
show_balance(account_id)

# moves money, high bar
elif action.choice == "approve_transfer":
if action.confidence > 0.85:
approve_transfer(account_id)
else:
ask_user_to_confirm()
```

Same model. Same call. Same answer.
Your risk tolerance lives in your code.

Why this works at all: the probabilities are calibrated

Jev is trained with Reinforcement Learning for Calibrated Decisions, which optimises the probabilities against outcomes rather than against human preference. Across a group of predictions, higher confidence really does mean higher accuracy. That is what makes a threshold meaningful instead of decorative.

Calibration is a property of groups, not of any single answer. A 0.95 can still be wrong. Thresholds shown are illustrative; tune them on your own data.

The answer tells you what. Confidence tells you whether to act. One threshold for the whole system is the mistake.

TypeSafe trains with a method they call **Reinforcement Learning for Calibrated Decisions (RLCD)**. Where RLHF optimises a model against human preference, which is a well-documented path to confident-sounding output, RLCD optimises the probabilities against outcomes. The result is calibration: across a group of predictions, the ones marked 0.9 really are correct about ninety percent of the time. That property is what makes a threshold worth writing:

```
action = response.answers["intent"]

if action.confidence < 0.5:
    # Genuinely unsure. Do not guess.
    route_to_human(user_message)
elif action.choice == "check_balance":
    # Read-only and recoverable. A low bar is fine.
    show_balance(account_id)
elif action.choice == "approve_transfer":
    if action.confidence > 0.85:
        confirm_then_execute(account_id)
    else:
        ask_user_to_confirm(account_id)
```

Notice there is no single threshold. Showing the wrong screen costs a user three seconds. Approving the wrong transfer costs them money. The bar for acting without a human rises with the consequences of being wrong, and where that bar sits is a product decision that now lives in your code rather than in a prompt.

If an intelligent system cannot express honest uncertainty, it cannot be trusted. Most LLM classifiers cannot. Asking one how sure it is produces another confident sentence. The caveat matters and TypeSafe states it plainly. Calibration is a property of groups, not of individual answers. A 0.95 can still be wrong. What you get is a dial that behaves predictably in aggregate, which is enough to build on, and considerably more than you had.

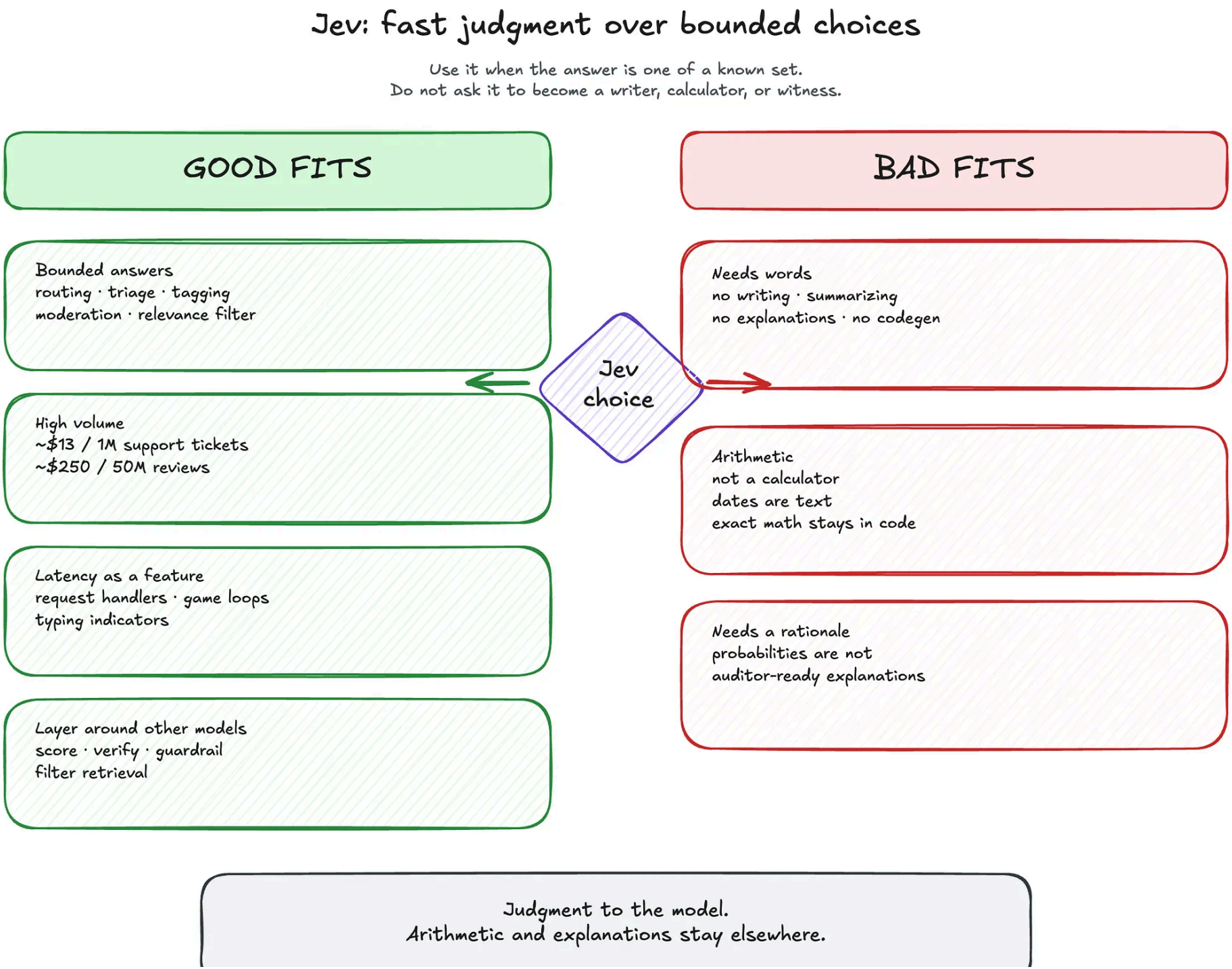
When should you use Jev instead of an LLM?

Use Jev when the answer space is bounded and known in advance, when volume or latency is the constraint, or when you need a calibrated confidence score to gate an action. Use an LLM when you need words, arithmetic, or an explanation.

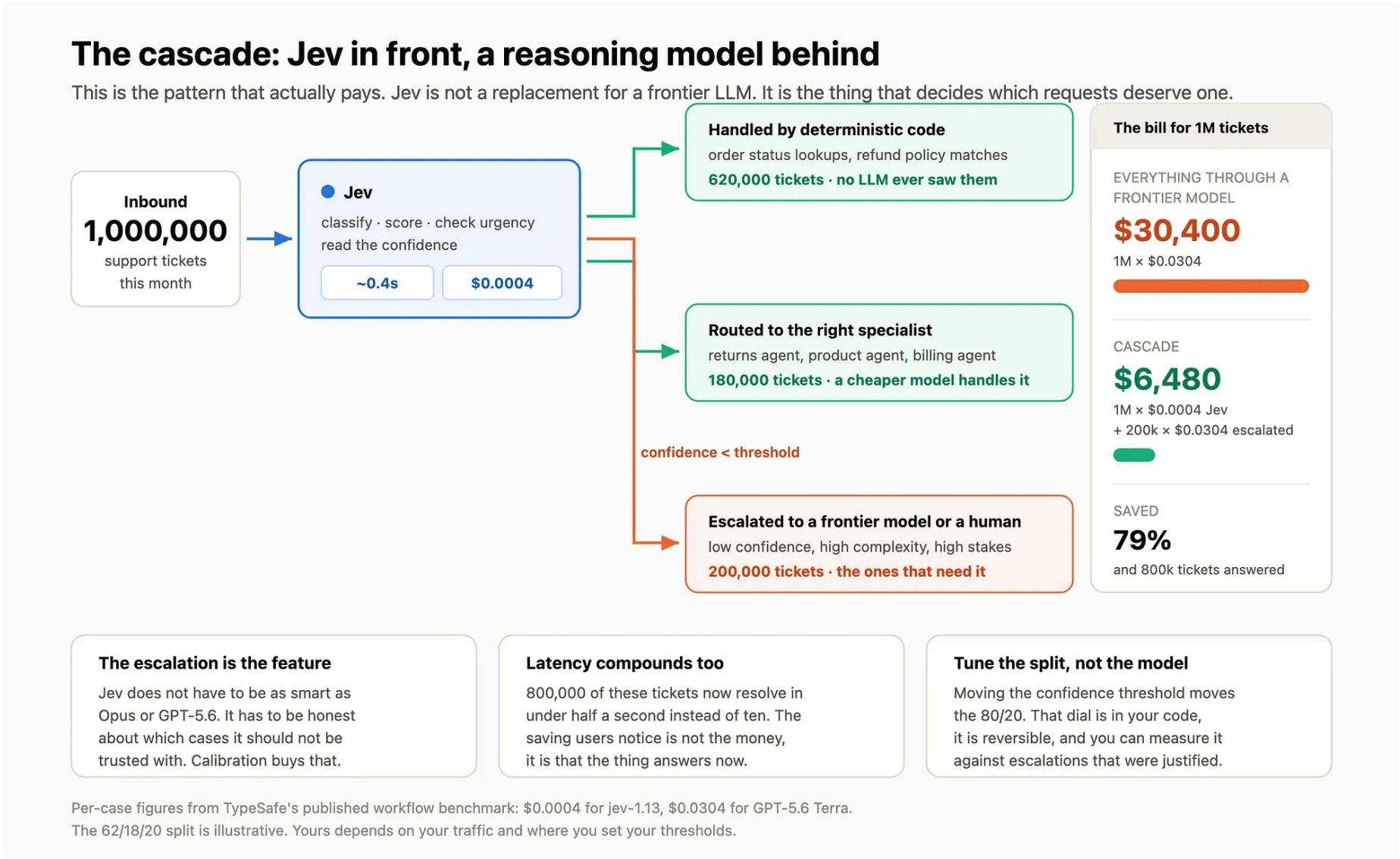
Jev is not competing with Opus 5 or GPT-5.6. It competes with the classifier you built out of a cheap LLM and four hundred lines of parsing, and with the

decisions you never automated because the economics did not work.

There are good fits & bad fits, so look at the diagram below & know when to use it. Do not fall into the trap of the “one size fits all” approach.


Jev: fast judgment over bounded choices
Use it when the answer is one of a known set.
Do not ask it to become a writer, calculator, or witness.
GOOD FITS
BAD FITS
Bounded answers
routing · triage · tagging
moderation · relevance filter
Needs words
no writing · summarizing
no explanations · no codegen
Jev choice
High volume
~$13 / 1M support tickets
~$250 / 50M reviews
Arithmetic
not a calculator
dates are text
exact math stays in code
Latency as a feature
request handlers · game loops
typing indicators
Needs a rationale
probabilities are not
auditor-ready explanations
Layer around other models
score · verify · guardrail
filter retrieval
Judgment to the model.
Arithmetic and explanations stay elsewhere.

The architecture that pays: the cascade



Jev is not a replacement for a frontier LLM. It is the thing that decides which requests deserve one.

Put those together and the interesting architecture is not **“replace your LLM.”** It is a cascade, with Jev as the fast, cheap, calibrated front door that decides what happens next.

Most requests get resolved by ordinary code, because Jev classified them into something deterministic. Some get routed to a specialist model with the right context loaded. The remainder, the genuinely hard and genuinely expensive minority, go to a frontier model or to a person.

The arithmetic on a million support tickets is not subtle. Everything through a frontier model at roughly three cents a case is **$30,400**. The cascade, paying $0.0004 for every ticket and frontier prices for the twenty percent that need it, lands near **$6,480**.

But the money is the less interesting half. Eight hundred thousand of those tickets now get answered in under half a second rather than in ten. The saving a user actually notices is that the thing responds immediately.

And the dial that sets the split is the confidence threshold, which lives in your code, is reversible, and can be measured against whether your

escalations turned out to be justified. You tune the system without touching the model.

Practical Solutions actually built with Jev

Jev shipped two days ago and a lot of things are already being built with it

Treat these as launch-week artefacts rather than production case studies. Most are a day or two old, several are explicitly experiments, and every number is self-reported by its author. What they are good for is showing the shape of problems Jev fits.

1. 1,018 research papers classified for 8 cents

1kpapers.com, by Hassan El Mghari

Hassan
@nutlope · **Follow**

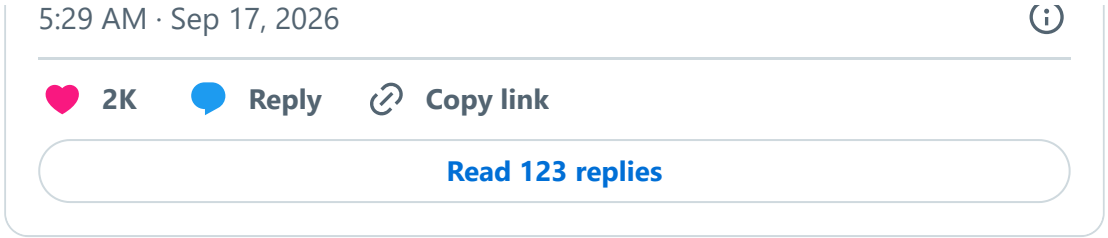
I used Jev to classify 1,018 AI research papers.

The result: $0.08 total cost and 256ms median end-to-end latency per paper.

The pipeline was:

1. Summarize each paper with DeepSeek V4 Flash
2. Send the title + summary + 24 possible topics to Jev
3. Use Jev to classify each Show more

Watch on X



2. A browser agent that books a flight in 7 seconds

browser-use/jev-ultrafast,

Gregor Zunic
@gregpr07 · **Follow**

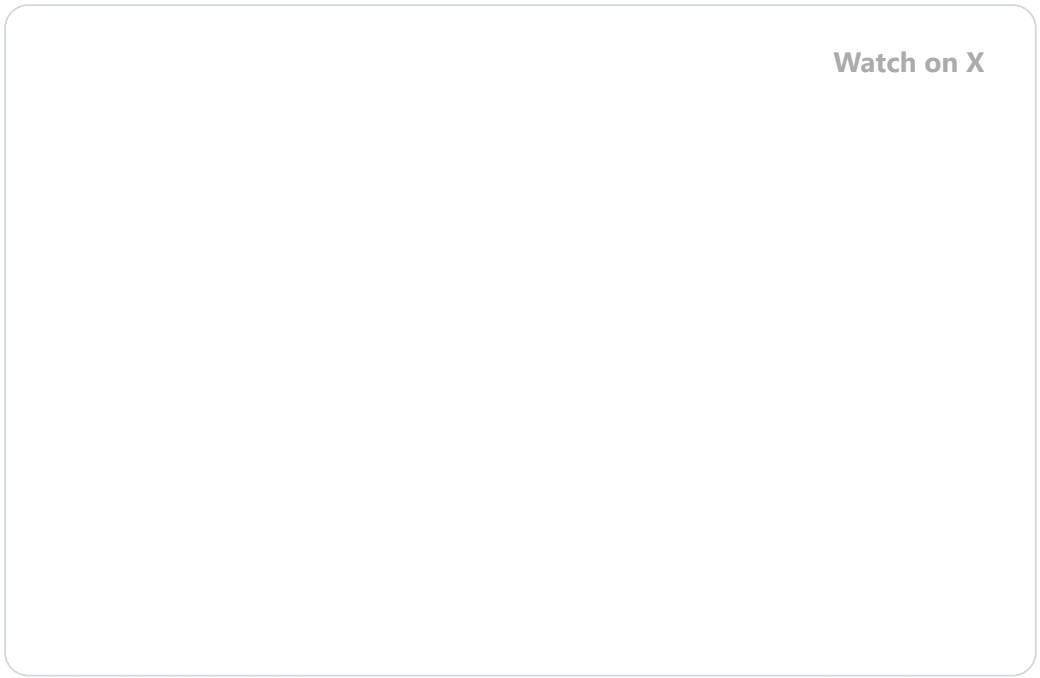
Breaking: Browser Use + Jev = Ultrafast ⚡

Findings flights took 7s and cost only $0.0039 🤯

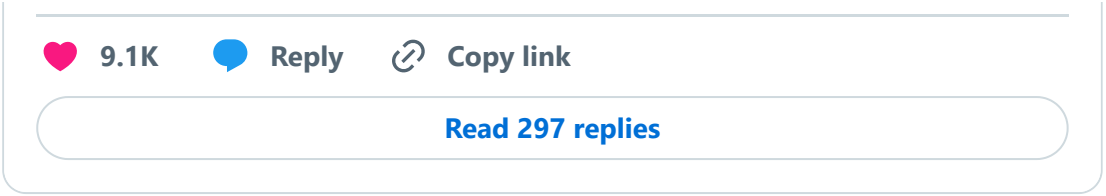
> new action space every step
> DOM state space
> small LLM fallback to type

(this video is at 1x speed btw)

Built a tiny open source browser agent. try it below ↓



4:26 AM · Sep 17, 2026



Zürich to London on real Google Flights in 7.1 seconds, for $0.0039, including text generation and page loads. The clever bit is speculative: the target questions for click, type and select are all asked in the same round trip, and only the one matching the chosen operation executes. Two decisions, one network call.

3. Computer use for a fiftieth of a cent per step

awlevin/typesafe-computer-use

Drives a Mac toward a plain-English goal without ever sending a screenshot to a large model. It reads the screen deterministically with OCR, asks Jev which action comes next, and calls a writing model only when a text field genuinely needs free text.

```
Jev            Claude Opus 5 (bare screenshot)
---------------------  -------------  --------------------------------
Cost per decision      $0.0002        $0.032
Cost per 12-step task  $0.003         $0.40 to $0.90
```

```
Model latency          0.13 to 0.38s  5.2s
End-to-end step        ~1.5s          ~5.5s
```

Jev read event dates off the pixels and compared them unaided, while the classifier needed explicit date parsing built around it. “Every piece of reasoning the frontier model does for free has to be rebuilt here as deterministic state.”

4. A market maker that decides every blockchain block

jarrodwatts/jev-trader

5. An autonomous drone with judgment at 2.5Hz

RomanSlack/jev-drone, 26 stars

A quadrotor flies a five-station obstacle course in MuJoCo using only its onboard camera. This one is worth studying because of how carefully it puts Jev in its place:

```
Rate     Layer                         Owner
-------  ----------------------------  ---------------------------------
500 Hz   Geometric flight controller   Ordinary code
50 Hz    Guidance and safety reflex    Ordinary code, always owns safety
15 Hz    Camera to symbolic scene      Classical CV
~2.5 Hz  Tactical judgment             Jev, advisory only
```

Classical computer vision compresses depth and segmentation into five forward range sectors, obstacle height and target bearing. Jev answers three questions in one call: a Choice over manoeuvres, a Score for risk, and a Noul asking whether the target is genuinely lost or just briefly occluded. As the README puts it, Jev “cannot be the perception layer, and it cannot run at control rate.”

6. Super Mario Bros from RAM, not pixels

fhshaik/typesafe-mario, 73 stars

An NES controller experiment where Jev picks one of seven legal actions. The model never sees a screenshot. The harness translates emulator telemetry and RAM into compact object-centric JSON: Mario’s motion, jump trajectory, upcoming enemies, terrain, measured response delay, recent

control results. It is a clean demonstration of the discipline Jev forces, which is that you must decide what the state actually is before you can ask about it.

Also worth a look

- phyous/tsai-sc has Jev complete Strongarm, the first combat mission of the original StarCraft shareware campaign, across 421 model decisions, with a published verification report.
- TypeSafe Typewriter by Steve Krouse is a live Val Town page that updates 16 typed judgments as you type. It is the fastest way to feel what sub-second structured judgment is like.
- Matija Sosic's 45-second explainer is, by some distance, the clearest short explanation of the core idea. It has 320,000 views because the official launch video left a lot of people confused.

The pattern across all of these is the same, and it is the lesson: ***keep the loop, the safety and the arithmetic in ordinary code, and use Jev for the narrow judgment in the middle that code finds hard to phrase.***

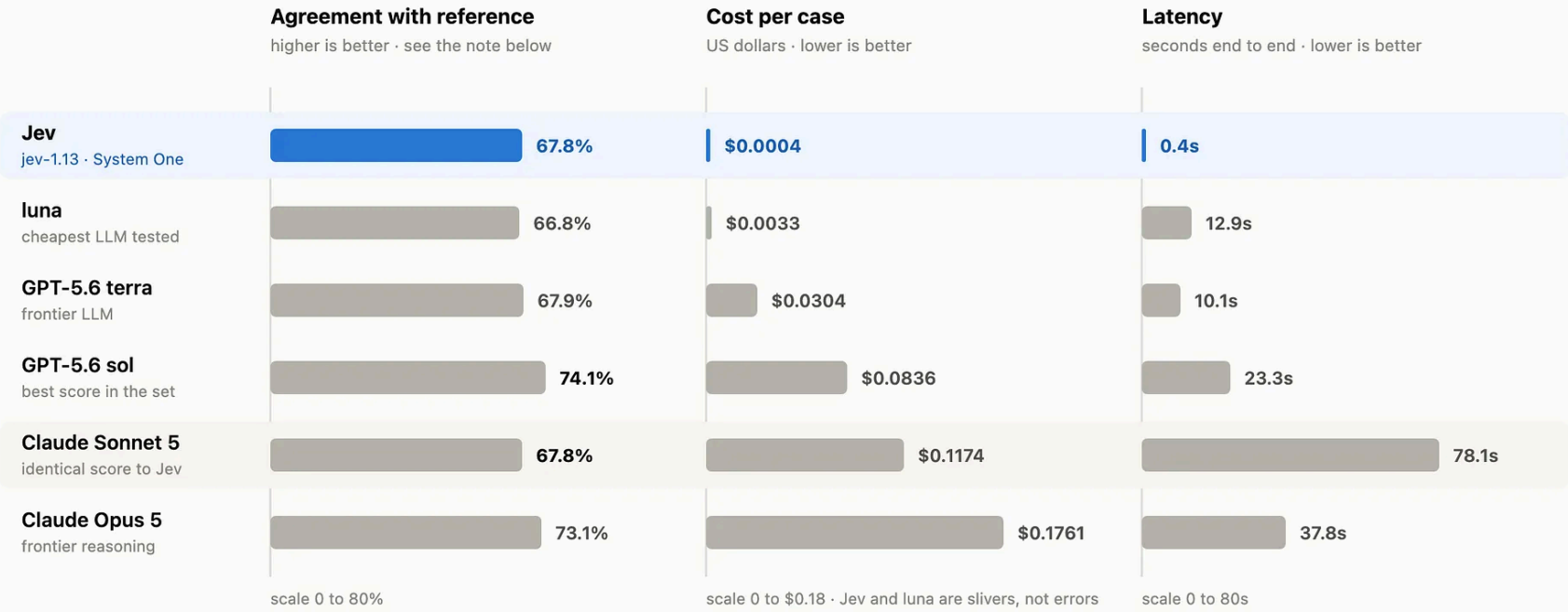
The model has documented rough edges, which TypeSafe publishes in an unusually candid page called “jaggedness.” Jev reads instructions literally

and answers the question you wrote rather than the one you meant. Accuracy degrades as you pack the state with material the question does not need. It does not treat state as potentially hostile, so text engineered to argue for its own classification can move the answer. If you are putting user-controlled content into state, that last one is yours to defend against.

One Important Thing to Note. Take this Seriously!

What the trade actually is

Jev matches mid-tier frontier models on these workflows. It trails the best by about six points. It does both at a fraction of the cost and latency.



The comparison that lands hardest

Sonnet 5 scores exactly what Jev scores: 67.8%.
It costs **293x** more per case and takes **195x** longer to say it.

Malformed outputs: 0%, but by construction

Jev's 0% is asserted from schema matching, not measured. LLM rates come from OpenRouter: 0.58% to 45.5% structured, 0.67% to 17.0% tool calls.

Read "agreement", not "accuracy". There is no ground truth here. TypeSafe scores every model against consensus labels built from GPT-6 Astra and Claude Fable 5.1 at high thinking, so those two models set the target and do not appear in the results. TypeSafe notes this biases toward OpenAI and Anthropic.

Four workflows: security incidents, agent trace observability, invoice processing, customer service. Built and run by TypeSafe. No independent reproduction exists yet.

Every model scored better inside the structured workflow than with the same policy as a single prompt. Haiku 4.5 went from 18.1% as a prompt to 53.6% in the workflow.

Every model in TypeSafe's evaluation scored better inside a structured workflow than with the same policy as a single prompt. Haiku 4.5 went from **18.1% as a prompt to 53.6% in the workflow**. Opus 5 went from 64.8% to 73.1%.

If you take nothing else from this research, take that decomposing a task into narrow typed judgments improves whatever model you are already paying for.

Frequently asked questions

What is Jev?
Jev is a frontier AI model from TypeSafe AI that returns typed, probabilistic decisions instead of generated text. It takes program state and typed questions and answers them in a single parallel pass in 70 to 500 milliseconds.

Who created Jev?
TypeSafe AI, founded in San Francisco in 2024 by Diogo Almeida (CEO),

Sasha Sheng (COO) and Erik Gafni (CTO). Almeida co-invented RLHF and InstructGPT at OpenAI and is credited as a contributor to ChatGPT and GPT-4.

How much does Jev cost?
$0.042 per million input tokens. Output tokens are free. TypeSafe's published workflow evaluation reports about $0.0004 per case, against $0.03 to $0.18 for frontier models.

Is Jev faster than GPT-5.6 or Claude Opus 5?
On TypeSafe's own benchmarks, yes, substantially: 0.4 seconds per case against 10.1s for GPT-5.6 Terra, 23.3s for Sol and 37.8s for Opus 5. These figures are self-reported and not independently reproduced.

Can Jev hallucinate?
It cannot return a value outside the schema you defined, which is guaranteed by construction rather than measured. It can still return the wrong valid value. Zero type errors is not zero mistakes.

Can Jev write text, code or summaries?
No. It is not trained to generate text at all. Use a generative model for that and Jev for the decision about what to do with it.

What are Choice, Score and Noul?

The three question types. Choice picks one option from up to 255. Score rates against two to ten ordered levels and can return a position between levels. Noul returns the probability that a yes/no statement is true.

How do I get access to Jev?

Early access is waitlisted at typesafe.ai. API keys come from the console at console.typesafe.ai. SDKs are typesafe-sdk for Python and @typesafe-ai/sdk for JavaScript.

What is RLCD?

Reinforcement Learning for Calibrated Decisions, TypeSafe's training method. Where RLHF optimises for human preference, RLCD optimises probabilities against outcomes, so higher confidence corresponds to higher accuracy across groups of predictions.

Should I replace my LLM with Jev?

No. The pattern that works is a cascade: Jev classifies and routes cheaply at the front, ordinary code handles what it can, and a frontier model or a human takes the hard minority.

Jev + Artificial Intelligence + Frontier Tech + Valyu +

Deep Research +



Written by unicodeveloper

5.6K followers · 541 following

Following

Developer & Technology Advocate. Writer of all things Technical & Magical. Software Craftsman. Artificial Intelligence. Life Sciences. Experience

Help Status About Careers Press Blog Store Privacy Rules Terms Text to speech